Machine Learning Session: Diabetes Classification

Logistic Regression, SVM, and KNN using caret in R

LUMUMBA W. VICTOR

Center for Data Analytics and Modeling
Chuka University

March 2026

Resource: https://cdam.chuka.ac.ke

Learning Objectives

By the end of this session, you will be able to:

1. Prepare health data for machine learning classification
2. Implement baseline logistic regression using glm()
3. Train models using caret::train() with cross-validation
4. Compare Logistic Regression, SVM, and KNN performance
5. Tune hyperparameters using grid search
6. Evaluate models using appropriate metrics
7. Interpret results for healthcare applications

Dataset: Diabetes.csv (Health Indicators)
Target: diabetes (Yes/No)

Session Outline

Part A: Data Prep (Slides 4-10)

1. Loading and Cleaning
2. Variable Types
3. Missing Values
4. Target Distribution
5. Train-Test Split
6. Preprocessing

Part B: Baseline glm() (Slides 11-18)

7. Why Start with glm()?
8. Fitting Baseline Model
9. Interpretation
10. Evaluation

Part C: caret::train() (Slides 19-40)

11. caret Overview
12. Logistic Regression
13. Support Vector Machines
14. K-Nearest Neighbors
15. Hyperparameter Tuning

Part D: Evaluation (Slides 41-50)

16. Model Comparison
17. Performance Metrics
18. Recommendations

Required Libraries

```r
# Core packages
library(caret)         # Unified ML interface
library(tidyverse)     # Data manipulation
library(pROC)          # ROC analysis
library(e1071)         # SVM engine
library(class)         # KNN engine
library(kernlab)       # Alternative SVM

# Theme settings
theme_set(theme_minimal())

# Set seed for reproducibility
set.seed(2026)
```

Note: Install packages if needed: `install.packages()`

Loading the Data

```r
# Read the dataset
diabetes_data <- read_csv("Diabetes.csv")

# Inspect structure
glimpse(diabetes_data)
summary(diabetes_data)

# Check dimensions
dim(diabetes_data)  # rows x columns

# Check for missing values
colSums(is.na(diabetes_data))

# Variable names
names(diabetes_data)
```

Variables:

- age, hypertension, heart_disease
- smoking_history, bmi
- HbA1c_level, blood_glucose_level
- diabetes (target)

Variable Types

```r
# Check variable classes
sapply(diabetes_data, class)

# Convert categorical variables to factors
diabetes_data <- diabetes_data %>%
  mutate(
    hypertension = factor(hypertension,
                          levels = c(0, 1),
                          labels = c("No", "Yes")),
    heart_disease = factor(heart_disease,
                           levels = c(0, 1),
                           labels = c("No", "Yes")),
    smoking_history = factor(smoking_history),
    diabetes = factor(diabetes,
                      levels = c(0, 1),
                      labels = c("No", "Yes"))
  )

# Verify conversions
str(diabetes_data)
```

Important: Factors required for classification in caret

Missing Values

```r
# Check missing values per variable
missing_summary <- diabetes_data %>%
  summarise(across(everything(),
                   ~sum(is.na(.))))
print(missing_summary)

# Option 1: Remove rows with missing values
diabetes_clean <- diabetes_data %>%
  na.omit()

# Option 2: Impute missing values
# library(mice)
# diabetes_imp <- mice(diabetes_data, m = 5)

# Check final dataset
cat("Original rows:", nrow(diabetes_data), "\n")
cat("Clean rows:", nrow(diabetes_clean), "\n")
cat("Removed:", nrow(diabetes_data) - nrow(diabetes_clean), "\n")
```

Document your choice: Removal vs. imputation affects results

Target Distribution

```r
# Class distribution
table(diabetes_clean$diabetes)
prop.table(table(diabetes_clean$diabetes))

# Visualize
ggplot(diabetes_clean, aes(x = diabetes)) +
  geom_bar(fill = "steelblue") +
  labs(title = "Diabetes Class Distribution",
       y = "Count") +
  theme_minimal()

# Calculate imbalance ratio
class_counts <- table(diabetes_clean$diabetes)
imbalance_ratio <- max(class_counts) / min(class_counts)
cat("Imbalance Ratio:", round(imbalance_ratio, 2), "\n")
```

If Imbalanced:

- Use appropriate metrics (AUC, F1, not just accuracy)
- Consider SMOTE or class weights

Exploratory Analysis

```r
# Summary by target
diabetes_clean %>%
  group_by(diabetes) %>%
  summarise(
    n = n(),
    mean_age = mean(age, na.rm = TRUE),
    mean_bmi = mean(bmi, na.rm = TRUE),
    mean_glucose = mean(blood_glucose_level, na.rm = TRUE)
  )

# BMI distribution by diabetes
ggplot(diabetes_clean, aes(x = bmi, fill = diabetes)) +
  geom_density(alpha = 0.4) +
  labs(title = "BMI Distribution by Diabetes Status")

# Age vs Glucose by diabetes
ggplot(diabetes_clean, aes(x = age,
                           y = blood_glucose_level,
                           color = diabetes)) +
  geom_point(alpha = 0.6) +
  labs(title = "Age vs Blood Glucose by Diabetes")
```

Goal: Understand patterns before modeling

Train-Test Split

```r
# Stratified split to preserve class balance
set.seed(2026)
train_index <- createDataPartition(
  diabetes_clean$diabetes,
  p = 0.70,       # 70% training
  list = FALSE    # return indices
)

# Split data
train_data <- diabetes_clean[train_index, ]
test_data  <- diabetes_clean[-train_index, ]

# Verify split
cat("Train:", nrow(train_data), "Test:", nrow(test_data), "\n")
prop.table(table(train_data$diabetes))
prop.table(table(test_data$diabetes))

# Save for later
write_csv(train_data, "diabetes_train.csv")
write_csv(test_data, "diabetes_test.csv")
```

Stratification: Ensures similar class proportions in both sets

Why Start with glm()?

Educational Value:

- Understand model mechanics without abstraction
- Interpret coefficients directly (log-odds, odds ratios)
- Build intuition before using automated tools
- Establish performance baseline

Practical Benefits:

- Fast to fit, no tuning required
- Highly interpretable results
- Serves as performance baseline
- Easy to debug and understand failures

Philosophy: Master fundamentals before automation

Logistic Regression Theory

Model Equation:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k$$

where $p = P(\text{diabetes} = \text{Yes})$

Key Properties:

- Output: Probability between 0 and 1
- Coefficients: Log-odds change per unit predictor change
- Odds Ratio: e^{β_j} = multiplicative effect on odds
- Estimation: Maximum Likelihood (not least squares)

Assumptions:

- Linearity in log-odds
- Independence of observations
- No perfect multicollinearity
- Large sample for reliable inference

Fitting Baseline Model with glm()

```r
# Fit logistic regression using base R glm()
glm_model <- glm(
  diabetes ~ age + hypertension + heart_disease +
             smoking_history + bmi + HbA1c_level +
             blood_glucose_level,
  data = train_data,
  family = binomial(link = "logit")
)

# View model summary
summary(glm_model)

# Extract key information
cat("Coefficients:\n")
print(coef(glm_model))

cat("\nAIC:", AIC(glm_model), "\n")
cat("Null Deviance:", glm_model$null.deviance, "\n")
cat("Residual Deviance:", glm_model$deviance, "\n")
```

family = binomial: Specifies logistic regression

Model Summary Interpretation

```r
# Detailed coefficient table
coef_table <- summary(glm_model)$coefficients
print(round(coef_table, 4))

# Interpret specific coefficients
bmi_coef <- coef(glm_model)["bmi"]
bmi_or <- exp(bmi_coef)  # Odds ratio
cat("\nBMI Coefficient:", round(bmi_coef, 3), "\n")
cat("Odds Ratio:", round(bmi_or, 3), "\n")
cat("Interpretation: Each 1-unit BMI increase multiplies odds of",
    "diabetes by", round(bmi_or, 2), "\n")

# Glucose coefficient
glucose_coef <- coef(glm_model)["blood_glucose_level"]
glucose_or <- exp(glucose_coef)
cat("\nGlucose OR:", round(glucose_or, 3), "\n")
```

Reading the Output:

- Estimate: Log-odds change per unit
- Std. Error: Uncertainty in estimate
- z value: Test statistic

Predictions from glm()

```r
# Predict probabilities on test set
glm_probs <- predict(glm_model,
                     newdata = test_data,
                     type = "response")

# Convert to class predictions (threshold = 0.5)
glm_pred_class <- ifelse(glm_probs > 0.5, "Yes", "No")
glm_pred_class <- factor(glm_pred_class,
                         levels = c("No", "Yes"))

# Create results dataframe
glm_results <- data.frame(
  Observed = test_data$diabetes,
  Predicted = glm_pred_class,
  Probability = glm_probs
)

# First 10 predictions
head(glm_results, 10)
```

type = "response": Returns probabilities (not log-odds)

Confusion Matrix (glm Results)

```r
# Create confusion matrix
glm_cm <- confusionMatrix(glm_pred_class,
                          test_data$diabetes,
                          positive = "Yes")

# Print full results
print(glm_cm)

# Extract key metrics
cat("\nKey Metrics:\n")
cat("Accuracy:", round(glm_cm$overall["Accuracy"], 3), "\n")
cat("Sensitivity:", round(glm_cm$byClass["Sensitivity"], 3), "\n")
cat("Specificity:", round(glm_cm$byClass["Specificity"], 3), "\n")
cat("Balanced Accuracy:", round(glm_cm$byClass["Balanced Accuracy"], 3), "\n")
```

Confusion Matrix:

- TP: True Positives (correctly identified diabetes)
- TN: True Negatives (correctly identified no diabetes)
- FP: False Positives (false alarms)
- FN: False Negatives (missed cases - critical in healthcare)

ROC Curve and AUC (glm)

```r
# Create ROC object using pROC package
glm_roc <- roc(test_data$diabetes, glm_probs,
               levels = c("No", "Yes"), direction = "<")

# Plot ROC curve
plot(glm_roc,
     main = "ROC Curve: Baseline Logistic Regression",
     col = "steelblue", lwd = 2)
abline(a = 0, b = 1, lty = 2, col = "gray")

# Extract AUC
glm_auc <- auc(glm_roc)
cat("AUC:", round(glm_auc, 3), "\n")

# Confidence interval for AUC
ci_auc <- ci(glm_roc)
cat("95% CI for AUC:", round(ci_auc[1], 3), "-",
    round(ci_auc[3], 3), "\n")
```

AUC Interpretation: 0.5 = random, 0.7-0.8 = acceptable, ¿0.9 = excellent

Limitations of Manual glm() Approach

What We Did Manually:

- Preprocessing (scaling, encoding)
- Train-test split
- Cross-validation (not done yet!)
- Hyperparameter tuning (none for basic glm)
- Model comparison framework

Challenges:

- Reproducibility: Many manual steps to track
- Scalability: Hard to compare multiple models
- Tuning: No built-in grid search for glm
- Preprocessing: Risk of data leakage if not careful
- Metrics: Manual calculation of advanced metrics

Solution: Use caret::train() for unified, automated workflow

caret Package Overview

caret: Classification And REgression Training

Core Functions:

- `train()`: Fit model with preprocessing and resampling
- `trainControl()`: Define resampling strategy
- `preProcess()`: Data transformation utilities
- `confusionMatrix()`: Classification evaluation
- `varImp()`: Variable importance extraction

Key Advantages:

- Consistent syntax across model types
- Automatic handling of categorical predictors
- Built-in parallel processing support
- Extensive documentation and examples

Documentation: https://topepo.github.io/caret/

train() Function Syntax

```r
# Basic train() syntax
model <- train(
  formula,                # e.g., diabetes ~ .
  data,                   # Training dataset
  method,                 # Model type (e.g., "glm")
  trControl,              # trainControl object
  preProcess,             # Preprocessing steps
  tuneGrid,               # Hyperparameter grid (optional)
  metric,                 # Metric to optimize (e.g., "ROC")
  ...                     # Model-specific arguments
)

# Example structure for logistic regression
log_caret <- train(
  diabetes ~ .,
  data = train_data,
  method = "glm",
  trControl = cv_control,
  preProcess = c("center", "scale")
)
```

method argument: Determines which model engine to use

Cross-Validation Setup

```r
# Define 10-fold cross-validation
cv_control <- trainControl(
  method = "cv",            # cross-validation
  number = 10,              # 10 folds
  savePredictions = "final",
  classProbs = TRUE,        # needed for ROC/AUC
  summaryFunction = twoClassSummary,
  verboseIter = FALSE
)

# Alternative: Repeated CV for more stable estimates
# cv_control <- trainControl(
#   method = "repeatedcv",
#   number = 10,
#   repeats = 3,
#   ...
# )
```

twoClassSummary: Requires classProbs = TRUE and factor target

Logistic Regression with caret

```r
# Fit logistic regression using caret
log_caret <- train(
  diabetes ~ age + hypertension + heart_disease +
             smoking_history + bmi + HbA1c_level +
             blood_glucose_level,
  data = train_data,
  method = "glm",
  trControl = cv_control,
  preProcess = c("center", "scale"),
  family = binomial
)

# View results
print(log_caret)

# Extract key information
cat("Resampling Results (10-fold CV):\n")
print(log_caret$results)

cat("\nOptimal Metric:", log_caret$metric,
    "=", round(log_caret$results[[log_caret$metric]], 3), "\n")
```

Note: For glm, results show single row (no tuning grid)

Cross-Validated Results Interpretation

```r
# View detailed resampling results
resamples_log <- resamples(list(Logistic = log_caret))
summary(resamples_log)

# Extract CV metrics
cv_metrics <- log_caret$results
cat("Cross-Validated Metrics:\n")
cat("Accuracy:", round(cv_metrics$Accuracy, 3),
    "+/-", round(cv_metrics$AccuracySD, 3), "\n")
cat("Kappa:", round(cv_metrics$Kappa, 3),
    "+/-", round(cv_metrics$KappaSD, 3), "\n")
cat("ROC:", round(cv_metrics$ROC, 3),
    "+/-", round(cv_metrics$ROCSD, 3), "\n")

# Compare with manual glm results
cat("\nComparison with Manual glm:\n")
cat("Manual glm AUC:", round(glm_auc, 3), "\n")
cat("caret CV ROC:", round(cv_metrics$ROC, 3), "\n")
```

Key Insight: CV estimates are more realistic than single train-test split

Predictions from caret Model

```r
# Predict on test set (automatically uses preprocessing)
log_caret_pred <- predict(log_caret, newdata = test_data)
log_caret_probs <- predict(log_caret, newdata = test_data,
                           type = "prob")

# Create results dataframe
log_caret_results <- data.frame(
  Observed = test_data$diabetes,
  Predicted = log_caret_pred,
  Prob_No = log_caret_probs$No,
  Prob_Yes = log_caret_probs$Yes
)

# First 10 predictions
head(log_caret_results, 10)

# Confusion matrix
log_caret_cm <- confusionMatrix(log_caret_pred,
                                test_data$diabetes,
                                positive = "Yes")
print(log_caret_cm)
```

Convenience: caret automatically applies same preprocessing to test data

Variable Importance with caret

```r
# Extract variable importance
log_importance <- varImp(log_caret, scale = TRUE)
print(log_importance)

# Plot importance
plot(log_importance,
     main = "Variable Importance: Logistic Regression",
     col = "steelblue")

# Manual extraction for custom plotting
imp_df <- as.data.frame(log_importance$importance)
imp_df$Variable <- rownames(imp_df)
imp_df <- imp_df %>% arrange(desc(Overall))

# Top predictors
ggplot(head(imp_df, 10),
       aes(x = reorder(Variable, Overall), y = Overall)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Top Predictors of Diabetes",
       x = "Variable", y = "Importance (scaled)")
```

Interpretation:

Regularized Logistic Regression (Optional)

```r
# Use glmnet for L1/L2 regularization
log_glmnet <- train(
  diabetes ~ .,
  data = train_data,
  method = "glmnet",
  trControl = cv_control,
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(
    alpha = c(0, 0.5, 1),
    lambda = seq(0.001, 0.1, length = 10)
  ),
  family = "binomial"
)

# View tuning results
print(log_glmnet)
plot(log_glmnet)

# Extract best parameters
best_params <- log_glmnet$bestTune
cat("Best alpha:", best_params$alpha,
    "Best lambda:", best_params$lambda, "\n")
```

Model Comparison: glm vs caret

```r
# Compare performance metrics
comparison <- data.frame(
  Model = c("Manual glm", "caret glm"),
  Accuracy = c(
    glm_cm$overall["Accuracy"],
    log_caret_cm$overall["Accuracy"]
  ),
  Sensitivity = c(
    glm_cm$byClass["Sensitivity"],
    log_caret_cm$byClass["Sensitivity"]
  ),
  Specificity = c(
    glm_cm$byClass["Specificity"],
    log_caret_cm$byClass["Specificity"]
  ),
  AUC = c(
    as.numeric(glm_auc),
    cv_metrics$ROC
  )
)

print(round(comparison, 3))
```

Logistic Regression Summary

Key Takeaways:

- glm() provides transparent, interpretable baseline
- caret::train() automates preprocessing and validation
- Cross-validation gives more realistic performance estimates
- Variable importance helps identify key predictors
- Regularization (glmnet) can handle high-dimensional data

Performance Summary:

- Accuracy: XX% (depends on data)
- AUC: 0.XX (discrimination ability)
- Sensitivity: Critical for healthcare applications
- Interpretability: High (coefficients, odds ratios)

Next: Support Vector Machines for potentially better non-linear performance

SVM Theory Overview

Core Idea: Find hyperplane that maximizes margin between classes

Key Concepts:

- **Margin:** Distance between hyperplane and nearest points
- **Support Vectors:** Data points on margin boundaries
- **Kernel Trick:** Map data to higher dimension for non-linear separation
- **Soft Margin:** Allow some misclassifications (controlled by C)

Optimization Problem:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i$$

subject to $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i$, $\xi_i \geq 0$

Advantage: Effective in high dimensions, robust to outliers

Kernel Functions

Common Kernels in caret:

Kernel	Formula	Use Case
Linear	$K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$	Linearly separable
Polynomial	$K(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^T \mathbf{x}')^d$	Feature interactions
RBF (Radial)	$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \lVert \mathbf{x} - \mathbf{x}' \rVert^2)$	Most flexible
Sigmoid	$K(\mathbf{x}, \mathbf{x}') = \tanh(\alpha \mathbf{x}^T \mathbf{x}' + c)$	Neural network-like

Key Hyperparameters:

- **C (cost):** Regularization parameter (larger = harder margin)
- γ **(sigma):** RBF kernel width (larger = more complex)
- **degree:** For polynomial kernel

Default in caret: RBF kernel with automatic tuning of C and sigma

SVM Specification in caret

```r
# SVM with radial kernel (most common)
svm_spec <- list(
  method = "svmRadial",
  trControl = cv_control,
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(
    C = c(0.25, 0.5, 1, 2, 4),
    sigma = c(0.01, 0.05, 0.1, 0.5)
  ),
  metric = "ROC"
)

# Alternative: Use tuneLength for automatic grid
# svm_spec <- list(
#   method = "svmRadial",
#   trControl = cv_control,
#   tuneLength = 10
# )
```

Critical: SVM requires scaled features; caret handles this automatically

Fitting SVM Model with train()

```r
# Fit SVM using caret
svm_model <- train(
  diabetes ~ age + hypertension + heart_disease +
             smoking_history + bmi + HbA1c_level +
             blood_glucose_level,
  data = train_data,
  method = "svmRadial",
  trControl = cv_control,
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(
    C = c(0.25, 0.5, 1, 2, 4),
    sigma = c(0.01, 0.05, 0.1, 0.5)
  ),
  metric = "ROC"
)

# View results
print(svm_model)

# Plot tuning results
plot(svm_model)

# Extract best parameters
```

SVM Predictions and Evaluation

```r
# Predict on test set
svm_pred <- predict(svm_model, newdata = test_data)
svm_probs <- predict(svm_model, newdata = test_data, type = "prob")

# Confusion matrix
svm_cm <- confusionMatrix(svm_pred,
                          test_data$diabetes,
                          positive = "Yes")
print(svm_cm)

# ROC curve
svm_roc <- roc(test_data$diabetes, svm_probs$Yes,
               levels = c("No", "Yes"), direction = "<")
plot(svm_roc, col = "darkred", lwd = 2,
     main = "ROC Curve: SVM Model")
abline(a = 0, b = 1, lty = 2)
cat("SVM AUC:", round(auc(svm_roc), 3), "\n")

# Compare with logistic regression
cat("\nComparison:\n")
cat("Logistic AUC:", round(cv_metrics$ROC, 3), "\n")
cat("SVM AUC:", round(auc(svm_roc), 3), "\n")
```

SVM vs Logistic: When to Use Which?

Use Logistic Regression When:

- Interpretability is important (coefficients, odds ratios)
- Relationship is approximately linear in log-odds
- Dataset is small to moderate size
- Need probabilistic output with calibrated probabilities
- Baseline model for comparison

Use SVM When:

- Decision boundary is expected to be non-linear
- Number of features is large relative to samples
- Need strong generalization with limited data
- Interpretability is less critical than accuracy
- Data has clear margin separation

Practical Tip: Try both; use cross-validation to select best performer

SVM Limitations and Considerations

Limitations:

- Computationally intensive for large datasets
- Probabilistic output requires Platt scaling (not native)
- Less interpretable than linear models
- Sensitive to choice of kernel and hyperparameters
- Requires careful feature scaling

Best Practices:

- Always scale features before SVM
- Start with RBF kernel and default grid
- Use cross-validation for hyperparameter tuning
- Monitor training time for large datasets
- Consider linear SVM for very high-dimensional data

For Healthcare: Consider interpretability requirements when choosing SVM

SVM Variable Importance

```r
# Extract variable importance for SVM
svm_importance <- varImp(svm_model, scale = TRUE)
print(svm_importance)

# Plot
plot(svm_importance,
     main = "Variable Importance: SVM (Permutation)",
     col = "darkred")

# Note: Permutation importance measures performance drop
# when variable is shuffled
```

Interpretation Caveats:

- Permutation importance can be unstable with correlated predictors
- Not as directly interpretable as logistic regression coefficients
- Useful for ranking, not for causal inference

Alternative: Use SHAP values or LIME for model-agnostic interpretation

SVM Model Diagnostics

```r
# Check support vectors
cat("Number of support vectors:",
    length(svm_model$finalModel$index), "\n")
cat("Proportion of support vectors:",
    round(length(svm_model$finalModel$index)/nrow(train_data), 3), "\n")

# Check if model is sparse (good sign)
if(length(svm_model$finalModel$index)/nrow(train_data) < 0.3) {
  cat("Model is sparse - good generalization potential\n")
} else {
  cat("Many support vectors - may be overfitting\n")
}

# Check training time
cat("Training time:", round(svm_model$times["user"], 2), "seconds\n")
```

Diagnostic Checks:

- Support vector proportion: Lower = better generalization
- Training time: Monitor for scalability concerns
- Cross-validation performance: Check for overfitting

Warning: High proportion of support vectors may indicate overfitting

SVM Summary

Key Takeaways:

- SVM finds maximum-margin hyperplane for classification
- Kernel trick enables non-linear decision boundaries
- caret automates scaling, tuning, and validation
- RBF kernel with tuned C and sigma is flexible default
- Requires scaled features and careful hyperparameter selection

Performance Summary:

- Accuracy: XX% (compare with logistic)
- AUC: 0.XX (discrimination ability)
- Training time: Longer than logistic (due to grid search)
- Interpretability: Lower than logistic regression

Next: K-Nearest Neighbors for instance-based, non-parametric approach

KNN Theory Overview

Core Idea: "Similar inputs have similar outputs"

Algorithm:

1. For new point $\mathbf{x}_{new}$, find k closest training points
2. Assign class by majority vote (classification)
3. Or average target values (regression)

Key Characteristics:

- **Lazy Learner:** No explicit training; stores all data
- **Non-Parametric:** No assumptions about data distribution
- **Instance-Based:** Prediction depends on local neighborhood
- **Distance Metric:** Euclidean default; others available

Tradeoffs:

- Simple to understand and implement
- Computationally expensive at prediction time
- Sensitive to irrelevant features and feature scaling

Best For: Small to medium datasets with meaningful distance metrics

Distance Metrics and k Selection

Common Distance Metrics:

Metric	Formula
Euclidean	$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum (x_j - y_j)^2}$
Manhattan	$d(\mathbf{x}, \mathbf{y}) = \sum \lvert x_j - y_j \rvert$
Minkowski	$d(\mathbf{x}, \mathbf{y}) = (\sum \lvert x_j - y_j \rvert^q)^{1/q}$

Effect of k:

- **Small k (e.g., 1):** Complex boundary; high variance (overfitting)
- **Large k:** Smooth boundary; high bias (underfitting)
- **Odd k:** Avoids ties in binary classification

Selection Strategy:

- Use cross-validation to test $k \in \{1, 3, 5, \ldots, \sqrt{n}\}$
- Start with $k = \sqrt{n}$ as rule of thumb
- Consider domain knowledge about neighborhood size

Critical: KNN requires scaled features; distance is meaningless otherwise

KNN Specification in caret

```r
# KNN specification with tuning
knn_spec <- list(
  method = "knn",
  trControl = cv_control,
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(
    k = seq(1, 21, by = 2)
  ),
  metric = "ROC"
)

# Alternative: Use tuneLength for automatic search
# knn_spec <- list(
#   method = "knn",
#   trControl = cv_control,
#   tuneLength = 10
# )
```

Note: caret's knn uses the `class` package backend

Fitting KNN Model with train()

```r
# Fit KNN using caret
knn_model <- train(
  diabetes ~ age + hypertension + heart_disease +
             smoking_history + bmi + HbA1c_level +
             blood_glucose_level,
  data = train_data,
  method = "knn",
  trControl = cv_control,
  preProcess = c("center", "scale"),
  tuneGrid = expand.grid(k = seq(1, 21, by = 2)),
  metric = "ROC"
)

# View results
print(knn_model)

# Plot performance vs k
plot(knn_model)

# Extract best k
best_k <- knn_model$bestTune$k
cat("Optimal k:", best_k, "\n")
cat("Best ROC:", round(max(knn_model$results$ROC), 3), "\n")
```

KNN Predictions and Evaluation

```r
# Predict on test set
knn_pred <- predict(knn_model, newdata = test_data)
knn_probs <- predict(knn_model, newdata = test_data, type = "prob")

# Confusion matrix
knn_cm <- confusionMatrix(knn_pred,
                          test_data$diabetes,
                          positive = "Yes")
print(knn_cm)

# ROC curve
knn_roc <- roc(test_data$diabetes, knn_probs$Yes,
               levels = c("No", "Yes"), direction = "<")
plot(knn_roc, col = "darkgreen", lwd = 2,
     main = "ROC Curve: KNN Model")
abline(a = 0, b = 1, lty = 2)
cat("KNN AUC:", round(auc(knn_roc), 3), "\n")
```

KNN-Specific Considerations:

- Prediction time increases with training set size
- Sensitive to irrelevant features (consider feature selection)
- Works well when similar instances have similar labels

KNN Limitations and Best Practices

Limitations:

- Computationally expensive at prediction time
- Sensitive to irrelevant features and noise
- Requires feature scaling (distance-based)
- Struggles with high-dimensional data
- No explicit model; hard to interpret or debug

Best Practices:

- Always scale features before KNN
- Perform feature selection to remove irrelevant predictors
- Use dimensionality reduction (PCA) for high-dimensional data
- Consider weighted voting (inverse distance weighting)
- Use efficient search structures for large n

Rule of Thumb: If $p \gg n$ (many features, few samples), KNN likely underperforms

Final Model Comparison

```r
# Collect all test set predictions
all_results <- data.frame(
  Observed = test_data$diabetes,
  Logistic_glm = glm_pred_class,
  Logistic_caret = log_caret_pred,
  SVM = svm_pred,
  KNN = knn_pred
)

# Evaluate all models
model_names <- c("Logistic_glm", "Logistic_caret", "SVM", "KNN")
comparison_table <- data.frame(
  Model = model_names,
  Accuracy = sapply(model_names, function(m) {
    cm <- confusionMatrix(all_results[[m]],
                          all_results$Observed,
                          positive = "Yes")
    cm$overall["Accuracy"]
  }),
  Sensitivity = sapply(model_names, function(m) {
    cm <- confusionMatrix(all_results[[m]],
                          all_results$Observed,
                          positive = "Yes")
```

Model Selection Recommendations

For Healthcare Applications (Diabetes Prediction):

Priority	Recommended Model	Reason
Interpretability	Logistic Regression	Coefficients, odds ratios
Accuracy	SVM or KNN	Potentially better non-linear fit
Speed	Logistic Regression	Fast training and prediction
Small Data	Logistic Regression	Less prone to overfitting
Large Data	KNN (with indexing)	Scales better than SVM
Imbalanced Data	Logistic with threshold tuning	Control sensitivity/specificity

Our Recommendation:

- Start with logistic regression for baseline and interpretability
- If accuracy is insufficient, try SVM with RBF kernel
- Use KNN if data has clear local structure
- Always validate with cross-validation and hold-out test set

Golden Rule: No free lunch; choose model based on data and application needs

Practical Implementation Checklist

Before Modeling:

1. Define target variable clearly (avoid data leakage)
2. Handle missing values appropriately
3. Split data before any preprocessing
4. Scale features for distance-based models (SVM, KNN)
5. Check class balance and plan for imbalance if needed

During Modeling:

1. Start with simple baseline (logistic regression)
2. Use cross-validation for reliable performance estimates
3. Tune hyperparameters on validation set only
4. Monitor training time and computational resources
5. Save preprocessing parameters for deployment

After Modeling:

1. Evaluate on held-out test set (never used in training/tuning)
2. Report multiple metrics (not just accuracy)
3. Document model limitations and assumptions

Healthcare Application Considerations

Ethical Considerations:

- **Bias:** Ensure training data represents all demographics
- **Privacy:** Protect patient health information
- **Transparency:** Explain model predictions to clinicians
- **Accountability:** Human-in-the-loop for medical decisions

Deployment Considerations:

- **Latency:** KNN can be slow on large datasets
- **Scalability:** Logistic Regression is very fast
- **Monitoring:** Track model drift over time
- **Updates:** Retrain with new data periodically

Warning: ML models assist, not replace, clinical judgment

Course Summary

What We Covered:

1. Data preparation and preprocessing
2. Baseline logistic regression with glm()
3. caret::train() for unified workflow
4. Logistic Regression, SVM, and KNN implementation
5. Hyperparameter tuning with cross-validation
6. Model evaluation and comparison
7. Healthcare application considerations

Key Skills Developed:

- R programming for machine learning
- Model training and validation
- Performance metric interpretation
- Model selection for specific applications
- Ethical considerations in healthcare ML

Next Steps: Practice with your own datasets, explore ensemble methods

Best Practices

Practice Recommendations:

- Try different hyperparameter grids for each model
- Experiment with alternative preprocessing (e.g., PCA, feature selection)
- Compare with tidymodels workflow for modern alternative
- Apply to your own health datasets with appropriate ethics approval

Next Session: Ensemble Methods and Model Stacking
Topics: Random Forest, XGBoost, stacking with caret

"The best model is the one that solves your problem, not the most complex one."

Thank You!

Questions?

Instructor: Lumumba W.V.
Email: cdam@chuka.ac.ke — lumumbavictor172@gmail.com
Phone: +254 706 038 599
Department: Center for Data Analytics and Modeling
Institution: Chuka University, Kenya

Resources:

- Course Materials: `https://cdam.chuka.ac.ke`
- R Code Repository: `https://github.com/cdam-chuka/r-sessions`
- caret Documentation: `https://topepo.github.io/caret/`
- Diabetes Dataset: Diabetes.csv (this session)